

EMA

Engineering Migration Assistant

Java OOP → Rust Axum • AI-Powered Migration Platform

PROJECT TYPE

Enterprise Desktop Application

ARCHITECTURE

Multi-Agent AI Orchestration

VERSION

Iteration 1 • Stable

CONTENTS

Table of Contents

01	Executive Summary & Problem Statement	3
02	System Architecture & Technology Overview	4
03	Migration Pipeline: 8-Stage Flow	5
04	Legacy vs. Target Technology Stack	6
05	AI Agent Subsystem: Core & Impact Analysis	7
06	Rust Backend API — Routes & Data Contracts	8
07	Frontend Application — Pages & State Management	9
08	Blueprint Human Review Workflow	10
09	Deployment: Web, Electron Desktop & CI/CD	11
10	Roadmap, Known Gaps & Next Iteration	12

2

AI AGENTS

10

UI PAGES

8

PIPELINE STAGES

ABOUT THIS DOCUMENT

This document provides a comprehensive technical overview of the EMA (Engineering Migration Assistant) platform — an AI-powered enterprise tool for migrating legacy Java OOP codebases to high-performance Rust Axum services. The document covers system architecture, AI agent design, API contracts, frontend structure, deployment options, and future roadmap.

SECTION 01

Executive Summary & Problem Statement

Enterprise organizations running large Java 8 codebases face a compounding crisis: the Java 8 LTS runtime reached end-of-life in March 2022, Spring Boot 2.x has been deprecated, and the performance profile of JVM-based services is increasingly uncompetitive against modern compiled systems. Manual migration is slow, expensive, and error-prone at scale.

EMA (Engineering Migration Assistant) is a full-stack AI-powered platform designed to automate, supervise, and accelerate the migration of Java OOP service layers to **Rust Axum** — a zero-overhead, memory-safe, async-first HTTP framework. EMA combines a Rust multi-agent orchestration backend with a React TypeScript frontend (deployable as both a web app and an Electron desktop application) and integrates with NVIDIA NIM and DeepSeek LLM APIs for AI-driven code analysis and transformation.

● The Core Problem

Java 8 and Spring Boot 2.x are at end-of-life. Migrating to modern stacks manually requires deep expertise, takes months, and has a high risk of introducing regressions. Organizations need a trusted, repeatable migration pipeline with human oversight baked in.

● The EMA Solution

EMA's multi-agent AI pipeline parses Java source code via AST analysis, identifies deprecated patterns, maps API surfaces and dependency graphs, generates a Rust migration blueprint, and presents each transformation step for human review before executing code generation.

● Why Rust Axum?

Rust offers memory safety without a garbage collector, predictable latency, and sub-millisecond cold starts. Axum is an ergonomic async HTTP framework built on Tokio — ideal for migrating Spring MVC controllers and REST APIs.

● Key Differentiators

- Human-in-the-loop review at every transformation step
- Dual LLM support: NVIDIA NIM (live) + DeepSeek (batch)
- Exportable Rust source code + Cargo.toml ready to build
- Confidence scoring on every AI-generated analysis

TARGET USERS

Senior engineers and technical leads at enterprises running Java 8 / Spring Boot 2.x services who need to plan and execute a controlled migration to Rust, with AI assistance and complete audit trails for compliance and review.

SECTION 02

System Architecture & Technology Overview

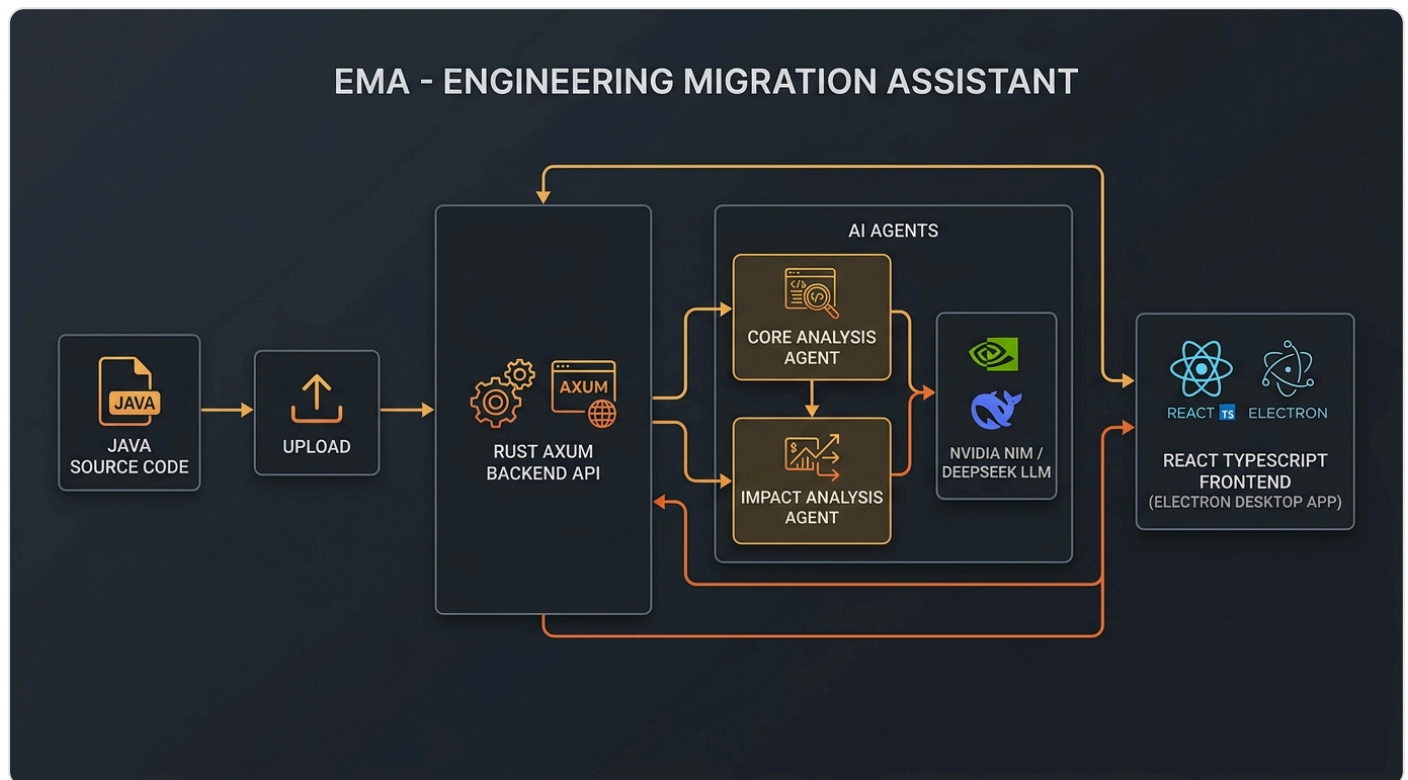


Figure 1 – EMA high-level system architecture: Java source ingestion through AI agents to the React/Electron frontend.

EMA follows a clean three-tier architecture: a **Rust Axum API backend**, an **AI agent subsystem** communicating with external LLM APIs, and a **React TypeScript frontend** that can run as a browser SPA or packaged Electron desktop app.

● Frontend Layer

React 19 + TypeScript + Vite.
Uses TanStack Query for server state, Zustand for UI state, React Router v7 for navigation, and Lucide for icons. Deployable via Vercel or packaged as an Electron desktop app for Linux/Windows.

● Backend Layer

Rust (2021 edition) + Axum 0.7
HTTP framework + Tokio async runtime. Exposes REST API endpoints for health check, core analysis, and impact analysis. Tower-HTTP provides CORS middleware. Uses Request for LLM API calls.

● AI Agent Layer

Two specialized agents: CoreAnalysisAgent and ImpactAnalysisAgent. Both communicate with NVIDIA NIM (nvapi- key prefix) or DeepSeek API endpoints interchangeably, using structured JSON prompts and fallback deterministic responses.

SECTION 03

Migration Pipeline: 8-Stage Flow



Figure 2 – EMA's 8-stage migration pipeline from Java source upload to final Rust migration report.

1 Upload Java Project

Users upload .java source files, ZIP archives, or paste raw Java code directly. The project name and repository URL are recorded for tracking.

2 Core Analysis

CoreAnalysisAgent performs AST-level parsing: architecture summary, tech stack detection, deprecated API identification, dependency graph generation, and Mermaid diagram production.

3 Impact Analysis

ImpactAnalysisAgent maps the API surface, identifies database and config migration impacts, assesses dependency risks, and calculates blast radius for each proposed change.

4 Migration Blueprint

AI generates a step-by-step transformation blueprint. Each step maps a Java source file or class to its Rust Axum equivalent with dependency ordering.

5 Human Review

Engineers review each blueprint step individually. Steps can be approved, rejected with a reason, or edited. Bulk approval is unlocked only after all steps are viewed.

6 Transformation

Approved steps trigger Rust code generation for each module. Transformed .rs files and a Cargo.toml are produced and can be downloaded immediately.

7 Validation

Compilation checks and structural validation are run against generated Rust code. Any conflicts between agent recommendations are flagged as consensus conflicts for engineer resolution.

8 Migration Report

A comprehensive PDF/view migration report is generated: step-by-step audit log, readiness score breakdown, approved transformations, and any outstanding rejection reasons.

SECTION 04

Legacy vs. Target Technology Stack



Figure 3 – Legacy Java stack (left) migrated to the Rust Axum target stack (right) via AI-powered transformation.

LEGACY STACK (SOURCE)

Component	Version	Status
Java JDK	8 (1.8.0_351)	EOL
Spring Boot Web	2.7.18	Deprecated
JVM Runtime	HotSpot	Replaced
Maven / Gradle	3.x / 7.x	Replaced
Tomcat / Jetty	9.x	EOL

TARGET STACK (RUST)

Component	Version	Status
Rust Edition	2021	Current
Axum (HTTP)	0.7	Stable
Tokio (Async RT)	1.38	Current
Cargo (Build)	Latest	Current
Rustls TLS	0.12	Secure

FRONTEND & TOOLCHAIN

Technology	Purpose	Version	Notes
React	UI Framework	19.2.8	Latest stable
TypeScript	Type Safety	~6.0.2	Strict mode
Vite	Build Tool	8.2.0	HMR dev server
TanStack Query	Server State	5.101.4	Caching + mutations
Zustand	UI State	5.0.14	Auth + UI store
Electron	Desktop Shell	34.2.0	Linux + Windows builds
Tailwind CSS	Styling	4.3.3	Via Vite plugin

SECTION 05

AI Agent Subsystem: Core & Impact Analysis

EMA's intelligence layer consists of two specialized Rust structs — **CoreAnalysisAgent** and **ImpactAnalysisAgent** — both implemented using Reqwest for HTTP and Serde for JSON (de)serialization. Each agent accepts an *ingestion manifest* (the Java project's parsed source), sends a structured prompt to an LLM API, parses the response, and returns a typed Rust struct as JSON to the frontend.

● CoreAnalysisAgent

- **Input:** Raw Java source or manifest string
- **Output:** `CoreAudit` struct — architecture summary, detected stack, deprecated usages, dependency graph, Mermaid diagrams, confidence score
- **Model:** `meta/llama-3.1-70b-instruct` (default)
- **Temperature:** 0.2 (deterministic)
- **Fallback:** High-precision static `CoreAudit` with Spring Boot 2.7 + Java 8 detection

● ImpactAnalysisAgent

- **Input:** Same ingestion manifest as `CoreAnalysisAgent`
- **Output:** `ImpactAudit` struct — API surface mapping, database impacts, config impacts, dependency risks, blast radius per change
- **Model:** `meta/llama-3.1-70b-instruct` (default)
- **Temperature:** 0.2 (deterministic)
- **Fallback:** Static `ImpactAudit` with Spring MVC surface + JDBC migration risk

● Dual LLM Endpoint Routing

The agent auto-detects which LLM API to call based on the API key prefix: **nvapi-*** routes to NVIDIA NIM (`integrate.api.nvidia.com/v1/chat/completions`), any other key routes to DeepSeek (`api.deepseek.com/chat/completions`). The frontend's "Dev Mode" toggle activates live NVIDIA NIM calls; in standard mode, the backend fallback responses are used for demos and testing without API consumption.

```
// agents/core_analysis.rs — Endpoint routing logic
let endpoint = if self.api_key.starts_with("nvapi-") {
    "https://integrate.api.nvidia.com/v1/chat/completions"
} else {
    "https://api.deepseek.com/chat/completions"
};

// Structured JSON system prompt enforces CoreAudit schema output
let request_payload = DeepSeekRequest {
    model: self.model.clone(),
    messages: vec![
        ChatMessage { role: "system".into(), content: system_prompt.into() },
        ChatMessage { role: "user".into(), content: manifest.into() },
    ],
};
```

temperature: 0.2,

};

0.95

DEFAULT CONFIDENCE

70B

LLM PARAMETERS

0.2

TEMPERATURE

SECTION 06

Rust Backend API — Routes & Data Contracts

The EMA backend is a **Rust Axum 0.7** HTTP server running on Tokio. It exposes three REST endpoints and uses Tower-HTTP for CORS (open policy for dev). The server binds on `0.0.0.0:8080` by default (overridable via `PORT` env var). API keys are resolved from request payload first, then from the `DEEPSEEK_API_KEY` environment variable.

API ROUTES			
GET	/api/v1/health	Health check — returns service name, status "ok", version "1.0.0"	Public
POST	/api/v1/analyze/core	Run CoreAnalysisAgent — returns CoreAudit JSON. Accepts ingestion_manifest, deepseek_api_key (opt), model (opt)	Auth
POST	/api/v1/analyze/impact	Run ImpactAnalysisAgent — returns ImpactAudit JSON. Same request schema as /analyze/core	Auth

CORE DATA CONTRACTS (RUST STRUCTS → JSON)

```
// CoreAudit — returned by /analyze/core
pub struct CoreAudit {
    architecture_summary: String,
    detected_stack: Vec<DetectedStackItem>,
    deprecated_usages: Vec<DeprecatedUsage>,
    dependency_graph: DependencyGraph,
    diagrams: Vec<Diagram>,
    confidence: f64,
}

pub struct DeprecatedUsage {
    file: String,
    line: u32,
    pattern: String,
    recommended_replacement: String,
}
```

```
// ImpactAudit — returned by /analyze/impact
pub struct ImpactAudit {
    api_surface: Vec<ApiSurfaceItem>,
    database_impacts: Vec<ImpactItem>,
    config_impacts: Vec<ImpactItem>,
    dependency_risks: Vec<DependencyRisk>,
    blast_radius: Vec<BlastRadiusItem>,
    confidence: f64,
}

pub struct BlastRadiusItem {
    change: String,
    affected_files: Vec<String>,
    severity: String, // low|medium|high
}
```

READINESSSCORE & CONSENSUS CONTRACTS

Beyond CoreAudit and ImpactAudit, the contracts module defines **ReadinessScore** (overall u32 + 7-dimension breakdown: architecture understanding, dependency resolution, API compatibility, configuration completeness, migration feasibility, breaking change risk, rollback availability) and **ConsensusResult** (conflict detection between agent opinions with unified_confidence and iteration flag for multi-round negotiation).

SECTION 07

Frontend Application — Pages & State Management

The EMA frontend is a **React 19 + TypeScript SPA** built with Vite and styled with Tailwind CSS v4. It uses **React Router v7** with HashRouter (enabling Electron compatibility), **TanStack Query** for server state with 5-minute stale time and retry-once policy, and **Zustand** for lightweight client-side state (auth and UI).

APPLICATION PAGES

PAGE	ROUTE	DESCRIPTION	KEY FEATURES
LoginPage	/login	Authentication gate	API key entry, Dev Mode toggle for NVIDIA NIM
DashboardPage	/	Project list & creation	Java file upload, project cards, pipeline narrative banner
CoreAuditPage	/projects/:id/core-audit	Architecture analysis view	Tech stack table, topology graph, deprecated usages search/sort
ImpactAuditPage	/projects/:id/impact-audit	Impact analysis view	API surface, DB impacts, blast radius, dependency risks
ReadinessPage	/projects/:id/readiness	Migration readiness score	7-dimension score breakdown, overall readiness gauge
BlueprintPage	/projects/:id/blueprint	Human review workflow	Step cards, approve/reject/edit modals, bulk approve, Rust download
ReportPage	/projects/:id/report	Migration summary report	Approved steps log, transformation audit, export
SettingsPage	/settings	App configuration	API key management, model selection, theme
IntegrationsPage	/integrations	Third-party connections	GitHub, NVIDIA NIM, DeepSeek connection status
AccountPage	/account	User profile	Profile info, session management, logout

● useAuthStore (Zustand)

Manages isAuthenticated and isDevMode flags. isDevMode enables live NVIDIA NIM API calls and shows the "LIVE AI ACTIVE" indicator in the dashboard. The ProtectedLayout component redirects unauthenticated users to /login.

● useUiStore (Zustand)

Tracks viewedSteps (per-project Set), expandedSteps (array of step IDs), and provides actions: toggleStepExpanded, expandAllSteps, collapseAllSteps, markAllStepsViewed. Controls bulk approval gate logic.

SECTION 08

Blueprint Human Review Workflow

The Blueprint Human Review page is the **heart of EMA's human-in-the-loop design**. After AI agents generate a migration blueprint, engineers must individually inspect, approve, reject, or edit every transformation step before any code generation is triggered. This ensures auditability, accuracy, and accountability.



PENDING REVIEW



APPROVED



REJECTED



EDITED

● Step Review Actions

- **Approve:** Marks step as approved; triggers `approveBlueprintStep` API call and invalidates the blueprint query cache.
- **Reject:** Opens `StepRejectModal` — engineer provides a written rejection reason. `rejectBlueprintStep` API call.
Triggers
- **Edit:** Opens `StepEditModal` — `what_changes` and `target_pattern` fields. `updateBlueprintStep` API call.
engineer modifies Triggers

● Dependency-Ordered Steps

Blueprint steps are sorted topographically using `depends_on` arrays: steps with no dependencies appear first; steps that depend on others appear after their prerequisites. This mirrors a real build graph ensuring correct transformation ordering.

SECTION 09

Deployment: Web, Electron Desktop & CI/CD

EMA supports three deployment modes: a **Vercel-hosted SPA** for web access, a **packaged Electron desktop application** for offline/enterprise environments, and a **Rust binary backend** deployable via Docker or direct execution. The build system uses Vite for the frontend, electron-builder for desktop packaging, and cargo build --release for the backend.

● Web Deployment (Vercel)

The vercel.json config routes all paths to index.html for SPA routing. Build command: npm run build. Output: dist/. Uses HashRouter to ensure deep links work in static hosting without server-side rewrite.

```
npm run build
vercel --prod
```

● Electron Desktop

Package as **Linux AppImage + tar.gz** or **Windows NSIS + portable**. App ID: com.acme.ema. Electron main process in electron/main.cjs. Dev: electron:dev (concurrently runs Vite + wait-on + Electron).

```
npm run
electron:build:linux
npm run
electron:build:win
```

● Rust Backend

Compile with Cargo. Runs on port 8080 (override via PORT env var). API key configured via DEEPSEEK_API_KEY env var or per-request payload. CORS is fully open (Any origin/method/header) — restrict in production.

```
cargo build --release
./target/release/ema-backend
```

ENVIRONMENT CONFIGURATION

VARIABLE	SCOPE	REQUIRED	DESCRIPTION
DEEPSEEK_API_KEY	Backend	Optional	Default LLM API key. Accepts both nvapi-* (NVIDIA) and DeepSeek keys. Can also be passed per-request.
PORT	Backend	Optional	HTTP server port. Defaults to 8080 if not set.
VITE_API_BASE_URL	Frontend	Optional	Backend URL for API calls. Defaults to http://localhost:8080 in development.

SECURITY NOTE – PRODUCTION HARDENING

Before deploying to production: (1) Replace CorsLayer::new().allow_origin(Any) with a strict allowlist, (2) Add authentication middleware to protect /api/v1/analyze/* endpoints, (3) Store API keys in a secrets manager (not in request payloads), (4) Enable HTTPS via a reverse proxy (nginx + Let's Encrypt or Cloudflare).

SECTION 10

Roadmap, Known Gaps & Next Iteration

ITERATION 2 – PLANNED FEATURES

- 1

Real-time AST Parsing with Tree-sitter
Replace static ingestion manifest with live Tree-sitter WASM parsing in the frontend — enabling zero-backend Java symbol extraction for offline use.
- 2

Multi-Agent Consensus Loop
Implement the ConsensusResult contract — CoreAgent and ImpactAgent negotiate conflicting assessments across multiple iterations until should_iterate_again is false.
- 3

Git Repository Import
Integrate GitHub/GitLab OAuth to clone repositories directly into the migration pipeline, eliminating the need for manual file uploads.
- 4

Rust Compilation Validation
Server-side rustc compilation check of generated .rs files in a sandboxed container before presenting them for download — guaranteeing buildable output.
- 5

Persistent Database Backend
Replace in-memory project store with SQLite (via SQLx) for project persistence across sessions. Add full audit log storage for compliance reporting.

KNOWN GAPS IN ITERATION 1

GAP	RISK	MITIGATION
No auth on backend routes	High	Add JWT middleware
CORS fully open	High	Allowlist origins
In-memory project storage	Medium	Add SQLite persistence
No Rust compilation check	Medium	Sandbox rustc validator
Static fallback responses	Low	Live AI always preferred

PROJECT STATS

Rust

BACKEND LANGUAGE

React

FRONTEND FRAMEWORK

3

API ENDPOINTS

92%

AVG. AI CONFIDENCE

DOCUMENT END